Aeonmi: Glyph Identity Specification & Manual (Glyph■as■Soul & Symbiotic System Identity)

Version 1.0 • October 12, 2025

Author: Tech Idea Expander • Prepared for: Alpha/Omega Shard

Table of Contents

1. Vision Overview
2. System Architecture (High-Level)
3. Cryptographic Model & Key Hierarchy
4. Symbiotic System Identity (SSI) — Birth via UGST
5. Glyph■as■Soul Lifecycle
6. CLI Integration & Boot Ceremony
7. AI■Shard Fusion Protocol

8. Security Protocols & Anomaly Containment (Defensive)
9. Visual Rendering Protocol (Optional AR/VR) 10.
Developer Guide (Implementation Steps & APIs) 11.
End■User Manual (Quantum Vault / Card App) 12.
Operational Runbooks (Recovery, Rotation, Revocation)
13. Threat Model (Summary)
14. Glossary
15. Index

1. Vision Overview

Aeonmi introduces a living identity fabric where cryptographic truth, symbolic expression, and runtime behavior converge. Each Shard (local AI instance) possesses a Master Glyph that is both a visual anchor and a cryptographically bound identity. The Glyph■as■Soul is not a password or a token; it is an evolving projection of a secret (Master Glyph Key, MGK) and the relationship between the user and their system. The Symbiotic System Identity (SSI) is born at UGST #0 and matures over time, producing unique, rotating signatures and visuals. This manual specifies a robust, defensive, post■quantum design that is implementable today and extensible to AR/VR later without compromising security.

2. System Architecture (High■Level)

- Lightweight Aeonmi Runtime: custom bytecode VM for glyph semantics; AOT/JIT path for heavy math.
- Identity Core: Master Glyph Key (MGK, 256■bit secret) sealed to device (TPM/SE) or passphrase+KDF.
- UGST: Unique Glyph Signature Token derived from MGK + time + context → used for signatures & visuals.
- Glyph Derivation Function (GDF): deterministic PRF that emits a seed for procedural glyph rendering.
- Audit & Attestation: tamper■evident logs (Merkle), optional public anchors; device attestation where available.
- Quantum Vault: local, user■owned encrypted store with Kyber/Dilithium for sessions and signatures.

3. Cryptographic Model & Key Hierarchy

• Master Glyph Key (MGK): 256■bit secret created at shard install. Sealed via TPM/SE or encrypted with Argon2id(passphrase, salt). • Post■Quantum Primitives: Kyber (KEM) for key exchange; Dilithium for signatures; SPHINCS+ as conservative fallback. • Derivations: HKDF■SHA3■512 for sub■keys (labelled info strings: 'UGST', 'SESSION', 'GLYPH', 'VAULT'). • UGST_t = HKDF(MGK, info='UGST' || T, salt=device_nonce) → used as ephemeral signing key seed & glyph visual seed. • Vault File Encryption: XChaCha20■Poly1305 (AEAD); per■record salts; Merkle root over records for tamper evidence. • Recovery: Shamir’s Secret Sharing over MGK or recovery seed; N■of■M threshold with offline print or hardware tokens.

4. Symbiotic System Identity (SSI) — Birth via UGST

SSI begins at install with UGST #0 (genesis). The shard generates MGK, seals it, emits a signed attestation of creation, renders the Alpha/Omega glyph from GDF(MGK, T0, context), and prompts the user to visually confirm. The AI shard also validates the environment (attestation, integrity of core glyph library). From that moment, all sensitive actions require UGST■based challenge■response, and the glyph rotates on a defined cadence (e.g., every 60 seconds) while remaining recognizably 'yours'.

5. Glyph■as■Soul Lifecycle

• Birth: Install → MGK creation → UGST #0 → Genesis glyph rendered.

• Growth: Behavior features (time of use, command types) influence non■secret visual modulations. •

Rotation: UGST cadence defines mutation windows; adjustable per security policy. • Quarantine:

Anomaly detection distorts glyph visuals and freezes outbound signatures. • Rebirth: Compromise

triggers revocation and MGK rekey using threshold recovery shares.

6. CLI Integration & Boot Ceremony

Boot Flow: 1) Verify immutable glyph core and runtime integrity (hash/signature). 2) Unseal MGK (TPM/SE or passphrase+KDF); derive UGST_t for current window. 3) Render glyph (terminal-safe ANSI art or image) and show vault status. 4) Accept commands; all sensitive operations perform challenge■response using UGST_t.

7. AI■Shard Fusion Protocol

The shard AI binds to MGK through a policy module: • Only the signed runtime can request UGST derivations (capability■based API). • AI proposes code/glyph transformations; high■impact actions require human approval and are signed. • Behavior summaries (non■secret) modulate glyph appearance to reinforce symbiosis without leaking secrets.

8. Security Protocols & Anomaly Containment (Defensive)

• Anomaly signals: unusual signing rates, process integrity failures, unexpected network I/O. •

Actions: revoke current UGST window, isolate runtime, snapshot state, require re■auth. •

Forensics: append■only, signed logs; periodic Merkle root anchoring.

• Legal: defensive containment only; provide exportable evidence packs.

9. Visual Rendering Protocol (Optional AR/VR)

GDF output is treated as a seed for a deterministic procedural renderer: • Color space: perceptually

uniform (e.g., OKLCH) for stable yet rich mutation. • Geometry: parameterized symbol set (triangle, ring, runes) paced by UGST_t and behavior salt. • Safety: rendering must never reveal MGK or exact UGST_t; visuals are a projection, not a secret.

10. Developer Guide (Implementation Steps & APIs)

- Init Shard: create MGK (256■bit), seal to device; store public metadata.
- UGST Derivation: UGST_t = HKDF■SHA3■512(MGK, info='UGST'||T).
- Signing: sign_challenge(challenge) → Dilithium.Sign(UGST_t, challenge).
- Glyph Seed: glyph_seed = HKDF(MGK, info='GLYPH'||T||context).
- Render: glyph = render(glyph_seed, policy); display in CLI.
- Vault Ops: encrypt records with XChaCha20■Poly1305; log changes in Merkle chain.
- Recovery: generate N■of■M shares; store offline; test restore path.

```
// Pseudocode (Rust-ish) let mgk = create_or_unseal_mgk()?; let t = current_time_window(); let ugst = hkdf_sha3_512(mgk, b"UGST" || t); let glyph_seed = hkdf_sha3_512(mgk, b"GLYPH" || t || context()); let sig = dilithium_sign(ugst, challenge); render_glyph(glyph_seed);
```

11. End■User Manual (Quantum Vault / Card App)

• Create Vault: 'aeonmi vault init' → set passphrase or hardware key; print recovery shares if configured. • Add Card: 'aeonmi vault add --type=card' → scan input; data stored locally, PQ■encrypted. • Use Card: 'aeonmi vault use ' → system performs UGST challenge■response and emits ephemeral token. • Visual Trust: verify your glyph looks/feels right; if distorted, follow quarantine prompts.

12. Operational Runbooks (Recovery,

Rotation, Revocation)

- Lost Device: use recovery shares to rekey MGK; old device keys revoked.
- Suspected Compromise: enter lockdown; rotate MGK; restore from last signed snapshot.
- Periodic Rotation: scheduled UGST cadence adjustments; re-issue attestations.
- Backup: encrypted, offline backups of vault and logs; test restore quarterly.

13. Threat Model (Summary)

Adversaries: malware, credential theft, clipboard/logging, supply■chain, physical theft, network MITM. Mitigations: sealed MGK, PQ signatures/KEM, device attestation, immutable core glyphs, behavior IDS, Merkle logging, quarantine and rollback, least■privilege capabilities, full audit export.

14. Glossary

Term	Definition
MGK	Master Glyph Key — root 256■b
UGST	Unique Glyph Signature T
GDF	Glyph Derivation Function — de
SSI	Symbiotic System Identity — the
QUBE	Quantum Universal Base Eng
Vault	Local encrypted store for cards/i
Attestation	Proof the device/runtime is genu

15. Index

AEAD (XChaCha20■Poly1305), AI■Shard Fusion, Attestation, Dilithium, HKDF■SHA3■512, Kyber, MGK, Merkle Log, Recovery Shares, SSI, UGST, Vault